



Rapport de Projet : MLP et moteur d'autodiff appliqués à MNIST

Auteurs:

Jianye Shi, Hao, Xiang, Abdenmour

Encadrant:

Aurélien Delval

Responsables:

Aurélien Delval / Hugo Bolloré / Pablo Oliveira

December 16, 2025

Sommaire

1	Introduction	3
2	Guide de lecture	3
3	Description du projet	3
4	Implémentation	4
4.1	Choix techniques et justification	4
4.1.1	Implémentation	4
4.1.2	Matrix / Node / graphe de calcul	4
4.1.3	Architecture générale	4
4.2	Architecture du MLP et organisation du code	4
4.2.1	Couche de base : opérations numériques et graphe computationnel	4
4.2.2	Architecture du réseau	5
4.2.3	Module d'entraînement	5
4.3	Implémentation détaillée	5
4.3.1	Implémentation de Matrix	5
4.3.2	Construction du graphe computationnel (Node , MathOps)	6
4.3.3	Implémentation des couches du MLP	6
4.3.4	Implémentation de la loss et de l'optimisation	6
4.3.5	Boucle d'entraînement	7
5	Expérimentations (à rédiger après tests)	7
5.1	Configuration de l'environnement	7
5.1.1	Environnement Matériel / Hardware	8
5.1.2	Environnement Logiciel / Software	8
5.1.3	Configuration Spécifique et Protocole	8
5.2	Résultats expérimentaux	9
5.2.1	Passage à l'échelle (Thread Scaling)	9
5.2.2	Impact de l'Affinité sur les petites charges	9
5.2.3	Hiérarchie des performances et Speedup	9
5.2.4	Analyse de Profiling et Loi d'Amdahl : vers l'entraînement complet . . .	10
6	Analyse des performances (profiling)	10
7	Déroulé du Projet	10
8	Conclusion & perspectives	11
9	Références	11
10	Glossaire	11

1 Introduction

Ce document présente le rapport du projet de construction d'un réseau de neurones multicouche (MLP) et d'un moteur de différenciation automatique appliqué au jeu de données MNIST. Réalisé dans le cadre de l'UE Projet en première année de master de calcul haute performance, simulation à l'Université Paris Saclay, ce travail a pour objectif d'implémenter pas à pas les fondements d'un système d'apprentissage automatique sans dépendre de bibliothèques haut niveau.

L'objectif général du projet est double :

1. Comprendre et implémenter les mécanismes internes d'un réseau neuronal, incluant le passage avant, la rétropropagation et la mise à jour de paramètres.
2. Construire un moteur d'autodiff permettant de dériver automatiquement les expressions mathématiques du réseau, afin de calculer les gradients nécessaires à l'apprentissage.

En parallèle, le projet vise à familiariser les étudiants avec l'analyse expérimentale, l'optimisation du code et la préparation aux travaux de performance qui seront approfondis au second semestre.

2 Guide de lecture

Le rapport est structuré de manière à accompagner progressivement le lecteur :

- La première partie présente le contexte, le problème à résoudre et un bref état de l'art autour de MNIST et des MLP.
- La deuxième partie décrit en détail l'implémentation : choix de conception, organisation du code et intégration des différentes phases.
- La troisième partie expose les expérimentations réalisées sur MNIST ainsi que les observations associées.
- La quatrième partie propose une première analyse de performances (profiling) afin d'identifier les points critiques qui seront optimisés au second semestre.
- Un glossaire et des références sont placés en fin de document pour clarifier les notions techniques.

3 Description du projet

Le projet consiste à concevoir et implémenter un pipeline complet d'apprentissage supervisé comprenant :

- un module de manipulation de tenseurs,
- un réseau de couches linéaires avec fonctions d'activation,
- un moteur de différenciation automatique (autodiff),
- un mécanisme d'entraînement (optimiseur, fonction de perte),
- l'intégration du jeu de données MNIST et l'évaluation du modèle.

Ce développement a été réalisé en plusieurs phases successives décrites ci-dessous. Des détails complémentaires figurent dans le document d'état de l'art.

4 Implémentation

4.1 Choix techniques et justification

4.1.1 Implementation

Choix du langage C++ pour sa performance et son contrôle fin de la mémoire, en cohérence avec l'orientation *calcul haute performance* de l'UE. Implémentation d'un moteur de calcul et d'autodiff maison (Matrix, Node, MathOps) afin de comprendre explicitement la construction du graphe computationnel et le calcul des gradients, plutôt que de s'appuyer sur des bibliothèques haut niveau.

4.1.2 Matrix / Node / graphe de calcul

- Représentation des données numériques par une classe **Matrix** générique (doubles en 2D), offrant les opérations de base nécessaires au MLP (matmul, add, mul, transpose).
- Représentation du graphe computationnel par la classe **Node**, qui stocke la valeur courante, le gradient associé, les parents et une fonction de rétropropagation (`backwardFn_`).
- Utilisation d'un tri topologique (méthode statique `topoSort`) pour parcourir le graphe lors de la phase de backward, plutôt que d'utiliser une récursion profonde.

4.1.3 Architecture générale

Choix d'un perceptron multicouche (MLP) à base de couches linéaires plutôt que de réseaux convolutifs ou récurrents, afin de se concentrer sur les mécanismes d'autodiff et de rétropropagation. Introduction d'une interface **ActivationFunction** (implémentée par ReLU, Sigmoid, Tanh) pour permettre de changer de fonction d'activation sans modifier la structure globale du code. Encapsulation de la combinaison « couche linéaire + activation » dans la structure **LayerNode**, puis construction du modèle complet via un vecteur de **LayerNode** dans **MLPNetwork**.

4.2 Architecture du MLP et organisation du code

Cette section décrit l'architecture logicielle du projet ainsi que les relations entre les différents modules, telles que représentées dans le diagramme UML (Figure 1). L'objectif est de mettre en évidence la séparation des responsabilités, l'organisation hiérarchique des composants et le flux global des données au sein du système.

4.2.1 Couche de base : opérations numériques et graphe computationnel

La couche la plus fondamentale de l'architecture repose sur les classes **Matrix**, **Node**, **OperationNode** et **MathOps**. Elle constitue le socle sur lequel s'appuient toutes les composantes du réseau neuronal.

La classe **Matrix** fournit les structures de données nécessaires aux calculs numériques et implémente les opérations linéaires de base utilisées dans le réseau. La classe **Node** modélise les nœuds du graphe computationnel. Chaque nœud contient une valeur courante, un gradient associé, ainsi que des liens vers ses nœuds parents, ce qui permet de représenter explicitement les dépendances entre les opérations. La classe **OperationNode** étend **Node** afin d'identifier le type d'opération réalisée via un identifiant (`OpKind`), facilitant ainsi la lecture et l'analyse du graphe.

Les opérations mathématiques sont centralisées dans la classe **MathOps**, qui encapsule des fonctions telles que `matmul`, `add`, `mul`, `relu`, `sigmoid` ou `tanh`. Chaque appel à **MathOps** construit

un nouveau **Node**, définit ses dépendances et associe la fonction de rétropropagation correspondante. Cette organisation permet de découpler le moteur d'autodifférentiation de la définition du réseau, rendant le graphe computationnel générique et réutilisable.

4.2.2 Architecture du réseau

Au-dessus du moteur de calcul, l'architecture du réseau neuronal est construite à l'aide de modules de plus haut niveau, spécifiquement dédiés à la définition du modèle. La classe **LinearLayer** encapsule les paramètres entraînables du réseau, à savoir les poids et les biais, ainsi que l'opération de projection linéaire appliquée aux entrées.

Les fonctions d'activation sont modélisées via l'interface abstraite **ActivationFunction**, permettant une implémentation interchangeable des activations telles que ReLU, Sigmoid ou Tanh. L'unité fonctionnelle de base du réseau est la structure **LayerNode**, qui combine une couche linéaire et une fonction d'activation en un bloc cohérent. Le modèle complet est représenté par la classe **MLPNetwork**, structurée comme une séquence de **LayerNode**, ce qui permet de composer facilement des réseaux multi-couches de profondeur variable. Cette conception favorise la modularité et permet de modifier l'architecture du réseau sans impacter le moteur de calcul sous-jacent.

4.2.3 Module d'entraînement

Le module d'entraînement regroupe l'ensemble des composants nécessaires à l'apprentissage du modèle et à son évaluation. L'abstraction **LossFunction** permet de définir différentes fonctions de perte, dont **MSELoss** et **CrossEntropyLoss**, utilisées respectivement pour des tests simples et pour la classification sur MNIST. La mise à jour des paramètres est assurée par la classe abstraite **Optimizer**, avec une implémentation basée sur la descente de gradient stochastique (**SGDOptimizer**).

Les données sont fournies par **MNISTDataset**, puis organisées en mini-batches à l'aide de **DataLoader**. La classe **Trainer** coordonne l'ensemble du processus d'entraînement, en orchestrant le flux suivant : chargement des données, propagation avant dans le modèle, calcul de la perte, rétropropagation des gradients et mise à jour des paramètres. Cette séparation claire des rôles permet de faire évoluer indépendamment le modèle, la fonction de perte ou la stratégie d'optimisation, tout en conservant une architecture globale cohérente.

Figure 1: Diagramme de classes UML

4.3 Implémentation détaillée

Cette section décrit les principaux choix d'implémentation des classes et méthodes constituant le moteur de calcul, le réseau neuronal et le pipeline d'entraînement. L'accent est mis sur la logique interne des composants, plutôt que sur les détails syntaxiques du code.

4.3.1 Implémentation de Matrix

La classe **Matrix** constitue la brique de base du calcul numérique. Les données sont stockées sous forme d'un `vector<double>` contigu en mémoire, facilitant l'accès séquentiel et la manipulation directe des coefficients.

Les opérations fondamentales nécessaires au MLP sont implémentées explicitement, notamment :

- la multiplication matricielle (**matmul**), réalisée à l'aide d'une triple boucle $i-j-k$;
- les opérations élémentaires **add** et **mul** ;

- la transposition de matrices.

Ce choix d'implémentation privilégie la clarté et la maîtrise du calcul, au détriment d'optimisations bas niveau, qui seront étudiées dans une phase ultérieure du projet. Les paramètres du réseau sont initialisés aléatoirement dans une plage contrôlée afin de rompre la symétrie au démarrage de l'apprentissage.

4.3.2 Construction du graphe computationnel (Node, MathOps)

Le moteur d'autodifférentiation repose sur la classe `Node`, qui contient une valeur (`value_`), un gradient (`grad_`), une liste de nœuds parents et une fonction de rétropropagation (`backwardFn_`). Lorsqu'une opération est invoquée via `MathOps` (par exemple `matmul`), un nouveau `Node` est créé. Ses dépendances sont explicitement définies et une fonction `backwardFn_`, implémentée sous forme de lambda, est associée afin de calculer les gradients locaux.

La phase de rétropropagation s'effectue en trois étapes :

1. un tri topologique du graphe computationnel ;
2. l'initialisation du gradient au niveau de la fonction de perte ;
3. l'appel successif des fonctions `backwardFn_` de chaque nœud dans l'ordre inverse du passage avant.

Cette approche garantit une propagation correcte des gradients, tout en évitant les récursions profondes.

4.3.3 Implémentation des couches du MLP

LinearLayer La classe `LinearLayer` implémente la transformation affine du réseau. La méthode `forward(input)` réalise l'opération $\text{output} = \text{input} \times \text{weights} + \text{bias}$ à l'aide des opérateurs définis dans `MathOps`. Les paramètres sont initialisés via la méthode `randomInit()`, et la méthode `parameters()` permet d'exposer les nœuds entraînaibles au module d'optimisation.

Fonctions d'activation Les fonctions d'activation sont implémentées comme des surcouches appliquant les opérateurs correspondants du moteur (`relu`, `sigmoid`, `tanh`), ce qui permet de les intégrer naturellement dans le graphe computationnel.

LayerNode & MLPNetwork Un `LayerNode` combine une couche linéaire et une fonction d'activation en une unité cohérente. La classe `MLPNetwork` applique successivement ces blocs lors de la propagation avant, permettant de construire des réseaux multi-couches de manière flexible.

4.3.4 Implémentation de la loss et de l'optimisation

L'apprentissage du réseau repose sur l'association d'une fonction de loss et d'un algorithme d'optimisation, tous deux intégrés au moteur d'auto-différentiation. Les fonctions de loss sont définies via une interface abstraite `LossFunction`, qui impose la méthode `forward(pred, target)`. Celle-ci construit un nœud scalaire représentant la perte du batch, directement inséré dans le graphe de calcul.

Deux fonctions de loss ont été implémentées :

1. **MSELoss** : utilisée principalement pour les tests et la validation du moteur d'auto-différentiation. Elle correspond à l'erreur quadratique moyenne entre prédictions et cibles, la perte retournée étant la somme sur le batch, normalisée par le nombre de sorties.

2. **CrossEntropyLoss** : utilisée pour la classification MNIST. Elle opère directement sur les logits du réseau et intègre une opération de softmax stable numériquement. Le gradient par rapport aux logits s'exprime simplement comme la différence entre les probabilités prédites et les labels one-hot, ce qui garantit une rétropropagation efficace et stable.

L'optimisation des paramètres est assurée par une interface abstraite **Optimizer**, fournissant les méthodes **zeroGrad()** et **step()**. Un optimiseur SGD (*Stochastic Gradient Descent*) a été implémenté, mettant à jour chaque paramètre selon la règle $-\nabla L$. Cette séparation claire entre calcul des gradients et mise à jour des paramètres permet une architecture modulaire et facilement extensible vers d'autres méthodes d'optimisation.

4.3.5 Boucle d'entraînement

La boucle d'entraînement et d'évaluation est implémentée dans la classe **Trainer**. Elle repose sur une fonction centrale **runEpoch**, paramétrée par un indicateur permettant de distinguer les phases d'apprentissage et d'évaluation.

Au début de chaque époque, le **DataLoader** est réinitialisé afin de parcourir l'ensemble des mini-batches. Pour chaque batch, les matrices d'entrée et de cibles sont encapsulées dans des nœuds constants du graphe computationnel. La propagation avant est ensuite réalisée en appliquant successivement le modèle (**MLPNetwork**) aux données d'entrée, produisant les prédictions du réseau.

La fonction de perte est alors évaluée à partir des prédictions et des cibles, générant un nœud de perte généralement scalaire. La valeur numérique de la perte est accumulée afin de calculer une perte moyenne sur l'ensemble de l'époque. En parallèle, la précision (accuracy) est estimée au niveau du **Trainer** en comparant, pour chaque échantillon du batch, la classe prédite (argmax des logits) à la classe cible encodée en one-hot.

Lorsque l'époque est exécutée en mode entraînement, la phase de rétropropagation est déclenchée pour chaque batch. Les gradients des paramètres sont d'abord réinitialisés, puis la méthode **backward** est appelée sur le nœud de perte afin de propager les gradients dans le graphe computationnel. Les paramètres du réseau sont enfin mis à jour à l'aide de l'optimiseur. En mode évaluation, les étapes de rétropropagation et de mise à jour des paramètres sont désactivées, ce qui permet de mesurer les performances du modèle sans modifier son état interne.

À la fin de l'époque, la perte moyenne et la précision globale sont calculées à partir des quantités accumulées et retournées sous forme de métriques.

5 Expérimentations (à rédiger après tests)

5.1 Configuration de l'environnement

Toutes les expériences ont été réalisées sur une machine avec les spécifications suivantes :

5.1.1 Environnement Matériel / Hardware

Table 1: Configuration matérielle

Composant	Caractéristiques
Processeur	AMD Ryzen 9 8940HX with Radeon Graphics Architecture : x86_64 Cœurs / Threads : 16 physiques / 32 logiques (2 threads par cœur) Fréquence : 421.8 MHz (min) à 5386 MHz (max) Cache L1d : 512 KiB (16 instances) Cache L1i : 512 KiB (16 instances) Cache L2 : 16 MiB (16 instances) Cache L3 : 64 MiB (2 instances)
Mémoire	30 GiB de RAM système
Swap	8.0 GiB

Les expérimentations parallèles ainsi que les mesures d'énergie ont été réalisées sur cette même configuration matérielle.

5.1.2 Environnement Logiciel / Software

Table 2: Configuration logicielle

Composant	Version / Détails
Système d'exploitation	Fedora Linux 42 (Workstation Edition)
Compilateur C/C++	GCC 15.2.1 20250808 (Red Hat 15.2.1-1)
Python	Version 3.13.7
CMake	Version 3.31.6
Shell	zsh
IDE	Visual Studio Code on Linux

Les mesures de performance ont été réalisées à l'aide de `perf stat` avec les événements RAPL (`power/energy-pkg/` et `power_core/energy-core/`) pour quantifier la consommation énergétique. Les optimisations de compilation ont été testées avec les flags `-O3 -march=native` pour activer l'auto-vectorisation et les optimisations spécifiques à l'architecture.

5.1.3 Configuration Spécifique et Protocole

Afin de limiter la variabilité des résultats, les campagnes de mesure suivent un protocole systématique de verrouillage de fréquence et d'affinité CPU. Les huit premiers cœurs physiques sont forcés à 4.0 GHz à l'aide de `cpupower`, puis les exécutions `perf stat` sont liées explicitement à ce jeu de cœurs via `taskset`. Le script type est le suivant : Le détail des commandes (verrouillage de fréquence, événements suivi par `perf stat`, étapes de remise à l'état initial) est présenté en annexe ???. Ce protocole constitue l'adaptation AMD des commandes recommandées pour les processeurs Intel dans le Lab 6, lesquelles préconisent un verrouillage similaire sur quatre cœurs logiques avec des bornes de fréquence adaptées au turbo (par exemple `cpupower frequency-set -g performance -u 5000MHz -d 5000MHz`). Les deux approches ne diffèrent donc que par les limites matérielles et la taille du groupe de cœurs réservés à l'expérience. Les scripts d'analyse acceptent les variables `LOCK_FREQ` et `CPU_SET` afin de rejouer automatiquement ces réglages.

5.2 Résultats expérimentaux

Cette section détaille les résultats obtenus lors des campagnes de test, en analysant l’impact du parallélisme, de l’affinité et des optimisations bas niveau sur les performances.

5.2.1 Passage à l’échelle (Thread Scaling)

Nous avons évalué la scalabilité de notre implémentation OpenMP sur différentes tailles de matrices, en variant le nombre de threads de 1 à 16 sur nos 8 cœurs physiques.

Instabilité des petites matrices ($< 128 \times 128$). Lors des premiers tests pilotés par scripts Bash, nous avons observé une forte variabilité sur les petites tailles. Cette instabilité a été corrigée en déplaçant la boucle de mesures **à l’intérieur du programme C++**, éliminant ainsi le bruit lié au lancement de processus. Avec cette méthodologie rigoureuse (moyenne sur 100 itérations internes, warm-up de 5 runs), l’écart type est tombé à environ $2 \mu s$. Cependant, même avec une mesure stable, le multithreading dégrade les performances pour les matrices inférieures à 128×128 . Le surcoût de création et de synchronisation des threads (overhead Fork/Join) est supérieur au gain de calcul. Il est donc impératif d’utiliser `OMP_NUM_THREADS=1` pour ces cas.

Matrices moyennes et grandes ($\geq 512 \times 512$). Pour les tailles plus importantes, le gain du parallélisme devient net. Nous observons deux comportements distincts selon la charge thread :

- **8 threads (Saturation physique)** : C’est la configuration optimale sur notre machine. OpenMP utilise une attente active (*busy-wait*), ce qui maximise la réactivité mais rend le système sensible au bruit si d’autres processus tournent.
- **16 threads (Sursouscription)** : Lancer plus de threads que de cœurs physiques force l’OS à effectuer des changements de contexte (*context switching*). Pour une librairie hautement optimisée comme BLAS, dont les pipelines arithmétiques sont saturés, chaque interruption est coûteuse. Nous avons mesuré une perte de performance d’environ 40% avec BLAS en passant de 8 à 16 threads (0.05s contre 0.08s sur une matrice 2048×2048).

5.2.2 Impact de l’Affinité sur les petites charges

Le cas spécifique des matrices 28×28 (taille des images MNIST) a fait l’objet d’une étude d’affinité approfondie.

- **BLAS** : Le temps d’exécution reste constant à $\approx 0.81 \mu s$ quelle que soit la stratégie d’affinité, démontrant l’efficacité extrême de son micro-noyau séquentiel.
- **OpenMP** : La stratégie de placement des threads influe fortement sur les performances.
 - `OMP_PROC_BIND=close` : $2.53 \mu s$ (limite les communications inter-cœur).
 - `OMP_PROC_BIND=spread` : $4.43 \mu s$ (coût de communication accru).

Conclusion : Pour les couches d’entrée du réseau (784×1 ou 28×28), BLAS mono-thread est imbattable (3x plus rapide que le meilleur OpenMP).

5.2.3 Hiérarchie des performances et Speedup

La comparaison globale des implémentations sur grandes matrices met en évidence quatre paliers de performance (Figure ??) :

1. **Naïf (ijk)** : L’ordre des boucles provoque des accès mémoire non contigus (fautes de cache). Le temps explose en $O(N^3)$, atteignant plusieurs secondes pour $N = 2048$.

2. **Réordonné (ikj)** : La simple permutation des boucles améliore la localité spatiale et apporte un gain substantiel (Speedup $\approx 3.3\times$).
3. **OpenMP** : La parallélisation sur 8 cœurs offre un gain supplémentaire d'un ordre de grandeur (Speedup global $\approx 6.5\times$).
4. **BLAS (OpenBLAS)** : L'utilisation des instructions vectorielles (AVX2/FMA) et l'optimisation assembleur permettent un speedup final de plus de **1200x** par rapport à la version naïve sur les très grandes tailles.

5.2.4 Analyse de Profiling et Loi d'Amdahl : vers l'entraînement complet

Afin de comprendre l'impact de ces optimisations sur le temps total d'apprentissage, nous avons profilé une époque complète d'entraînement (forward + backward) avec **gprof**.

Avant optimisation (Kernel Naïf) : L'application est totalement limitée par le calcul (*compute-bound*). La fonction `Matrix::matmul` consomme **94.8%** du temps total (12.05s).

Après optimisation (BLAS) : Le temps passé dans `Matrix::matmul` s'effondre à moins de **1%** ($\approx 0.5s$). Le goulot d'étranglement se déplace alors vers :

- Le chargement des données (`MNISTDataset::load`) : $\approx 43\%$ du temps.
- Les allocations mémoire (`Matrix::Matrix`) : $\approx 29\%$ du temps.

Conséquence (Loi d'Amdahl) : Bien que le noyau de calcul soit accéléré de 1200x, l'accélération de l'application complète est plafonnée par les parties non optimisables (chargement, allocation). Le temps total d'une époque passe de 13.27s à 1.74s, soit un **speedup end-to-end de 7.6x**. Ce résultat démontre l'importance de considérer la chaîne complète : une fois le calcul optimisé, les efforts doivent se porter sur la gestion mémoire et les I/O pour obtenir de nouveaux gains.

6 Analyse des performances (profiling)

7 Déroulé du Projet

L'état de l'art a été rédigé conjointement par Jianye, Hao et Xiang, afin de présenter le contexte théorique du projet et les approches existantes en matière de réseaux neuronaux et de classification MNIST. L'équipe a ensuite organisé son travail en plusieurs phases successives, chacune étant portée par un ou plusieurs membres selon leurs affinités techniques.

La phase de conception a été initiée par Jianye, qui a rédigé la documentation préliminaire et produit la première version du diagramme UML, permettant de clarifier l'architecture générale du projet. La mise en place du premier prototype fonctionnel du MLP a ensuite été assurée par Abdennour, qui a développé les modules fondamentaux liés aux tenseurs, aux couches linéaires et aux fonctions d'activation.

La réalisation du moteur de différenciation automatique a été répartie entre Hao et Xiang, chacun prenant en charge une partie des opérations nécessaires au calcul du graphe et des gradients. Par la suite, Jianye a intégré l'ensemble de ces composants, généré une seconde version du diagramme UML pour refléter les ajustements apportés, et affiné la conception en fonction des problèmes rencontrés.

Le développement du mécanisme d'entraînement a été amorcé par Xiang, tandis que Abdennour et Hao poursuivent respectivement l'implémentation de l'optimiseur et des fonctions de perte. L'intégration du jeu de données MNIST et la validation du pipeline complet ont été assurées par Jianye.

Enfin, la rédaction du rapport est réalisée collectivement, de manière collaborative.

8 Conclusion & perspectives

9 Références

10 Glossaire